Rapport de Projet : Systèmes Temps Réel

Ordonnancement de tâches périodiques

Ton Nom

28 Avril 2026

Table des matières

1 Partie 1 : Analyse de la tâche τ_1 **2**
1.1 Description de la tâche 2
1.2 Résultats Statistiques 2
1.3 Interprétation du WCET 2

2 Partie 2 : Analyse d'ordonnançabilité **2**
2.1 Ensemble de tâches 2
2.2 Calcul du taux d'utilisation U 3

3 Partie 3 Simulation d'ordonnancement **3**
3.1 Stratégie et Complexité Algorithmique 3
3.2 Analyse des temps de réponse et des premières instances 3
3.3 Vérification théorique et stabilité 4
3.4 Simulation 2 : Optimisation et Sacrifice de τ_5 4
3.5 Analyse des résultats et conclusion 4

Introduction

Ce projet porte sur l'analyse et l'ordonnancement d'un ensemble de tâches dans un environnement temps réel. L'objectif est de créer une tâche de calcul de multiplication de grands entiers aléatoires(τ_1), d'évaluer la faisabilité d'un ensemble de sept tâches, et de proposer un algorithme d'ordonnancement non-préemptif optimisant le temps d'attente global.

1 Partie 1 : Analyse de la tâche τ_1

1.1 Description de la tâche

La tâche τ_1 consiste à calculer le produit de deux grands nombres entiers générés aléatoirement. Pour obtenir des mesures de temps fiables, nous avons exécuté cette tâche 10^7 fois en utilisant l'horloge système haute précision `clock_gettime`.

1.2 Résultats Statistiques

Après exécution du programme C et le tri des données via l'algorithme `qsort` de la librairie include <stdlib.h>, nous avons obtenu les résultats suivants (en nanosecondes) :

- **Minimum :** 37 ns
- **Premier Quartile (Q1) :** 40 ns
- **Médiane (Q2) :** 40 ns
- **Troisième Quartile (Q3) :** 41 ns
- **Maximum :** 106 206 ns

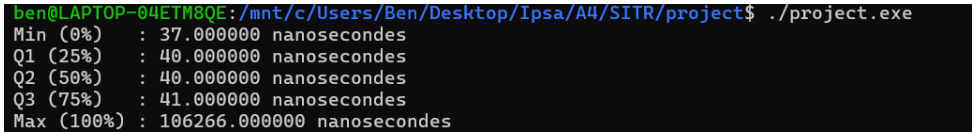

FIGURE 1 – Résultats statistiques de l'exécution de la tâche τ_1

1.3 Interprétation du WCET

Le *Worst-Case Execution Time* (WCET) retenu pour la suite de l'étude(après répétition de mon code est $C_1 = 1114000$ ns plus une marge soit $C_1 = 1$ ms (on arrondie pour la partie 2) . Cette valeur est critique car elle garantit que, même dans le pire scénario, la tâche τ_1 ne dépassera pas ce temps de calcul. Cette valeur est très grande car d'autres process tournent sur ma machine créeant ainsi des moments où la fonction sera en attente(c'est ce sur quoi nous travaillerons).

2 Partie 2 : Analyse d'ordonnançabilité

2.1 Ensemble de tâches

Le système est composé des tâches suivantes :

Tâche	C_i (ms)	T_i (ms)
τ_1	C_1	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

2.2 Calcul du taux d'utilisation U

Pour vérifier si le système est ordonnançable, nous calculons le taux d'utilisation du processeur :

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

Si $U \leq 1$, le processeur a la capacité de traiter l'ensemble des tâches.

$$U = 1/10 + 3/10 + 2/20 + 2/20 + 2/40 + 2/40 + 3/80 = 0.7475 << 1$$

Le taux d'utilisation n'est pas très élevé car notre fonction de produits de grands nombres est extrèmement rapide.

3 Partie 3 Simulation d'ordonnancement

3.1 Stratégie et Complexité Algorithmique

Pour vérifier l'ordonnançabilité en pratique, nous avons développé un simulateur en Python sur une période de 80 ms(qui est la plus petite période permettant de réveiller nos tâches un nombre entier de fois.

L'algorithme que l'on implémante est basé sur l'ordonnancement **EDF** (*Earliest Deadline First*), qui choisit toujours la tâche dont l'échéance est la plus proche. Afin de minimiser le temps d'attente global, nous avons ajouté la règle **SJF** (*Shortest Job First*) comme critère secondaire en cas d'égalité d'échéance(en effet nous avons des tâches dont l'échéance est la même. Concernant la complexité de notre algorithme, la sélection de la tâche à chaque instant t repose sur le tri de la liste des tâches prêtes. L'algorithme de tri natif de Python (*Timsort*) ayant une complexité de $O(N \log N)$ dans le pire des cas, la complexité de notre ordonnanceur à chaque décision est donc de $\mathcal{O}(N \log N)$, où N représente le nombre de tâches réveillées.

3.2 Analyse des temps de réponse et des premières instances

Pour chaque instance n de la tâche i, nous avons calculé le temps de réponse $R_i^n =$ Heure de fin $-$ Heure de réveil. Le tableau suivant détaille l'ordonnancement des premières instances de chaque tâche, jusqu'à la première exécution de τ_7.

Ce segment est représentatif car il montre que même lors de la phase la plus chargée (le démarrage à $t = 0$), le système traite la tâche la plus longue (τ_7) tout en respectant strictement chaque échéance.

Job	Reveil (a_{ij})	Debut	Fin	R_{ij} (ms)	Echeance (T_i)
$\tau_{1,1}$	0	0	1	1	10
$\tau_{2,1}$	0	1	4	4	10
$\tau_{3,1}$	0	4	6	6	20
$\tau_{4,1}$	0	6	8	8	20
$\tau_{5,1}$	0	8	10	10	40
$\tau_{1,2}$	10	10	11	1	20
$\tau_{2,2}$	10	11	14	4	20
$\tau_{6,1}$	0	14	16	16	40
$\tau_{7,1}$	0	16	19	19	80

3.3 Vérification théorique et stabilité

Conformément à la formule de récurrence du sujet :

$$R_{ij}^n = C_{ij}^n + \max(R_{ij}^{n-1} - (a_{ij}^n - a_{ij}^{n-1}), 0)$$

Nous avons vérifié sur l'intégralité des 29 jobs de la simulation que le terme $\max(R_{ij}^{n-1} - (a_{ij}^n - a_{ij}^{n-1}), 0)$ reste nul. Cela s'explique par le fait que chaque instance se termine avant l'arrivée de la suivante ($R_{ij}^{n-1} < T_i$). Il n'y a donc aucun cumul de retard, ce qui garantit la stabilité du système sur le long terme.

Enfin, concernant l'inactivité du processeur (*Idle Time*), la simulation complète confirme un total de 21 ms de temps libre sur l'hyperpériode de 80 ms. Notre algorithme étant non-préemptif et "work-conserving", ce temps d'inactivité est optimal.

3.4 Simulation 2 : Optimisation et Sacrifice de τ_5

Afin de tenter de minimiser davantage le temps d'attente global, nous avons modifié l'algorithme pour reléguer la tâche τ_5 au statut de tâche de fond (priorité minimale), l'autorisant explicitement à rater son échéance au profit des autres tâches.

— **Échéances ratées :** 0
— **Nouveau temps d'attente total :** 109.000 ms
— **Gain global :** -1.000 ms

3.5 Analyse des résultats et conclusion

Le temps d'attente global du système a augmenté de 1 ms, engendrant un gain négatif.

En effet, la minimisation du temps d'attente global est mathématiquement garantie par l'exécution prioritaire des tâches les plus courtes (SJF). Or, la tâche τ_5 possède un temps d'exécution relativement court ($C_5 = 2$ ms). En forçant artificiellement τ_5 à patienter en arrière-plan, le processeur privilégie l'exécution de tâches potentiellement plus longues (comme τ_2 ou τ_7 où $C = 3$ ms). Le fait de bloquer une tâche courte derrière une tâche longue dégrade structurellement le temps d'attente cumulé du système.

De plus, la charge globale du processeur étant modérée ($U \approx 74\%$), cette mise en attente forcée n'a même pas été suffisante pour faire rater son échéance à τ_5, dont la période de 40 ms offre une marge temporelle confortable. Nous pouvons donc conclure que pour cette configuration précise, sacrifier τ_5 n'est pas une stratégie d'optimisation pertinente.